

摘要：

DeepSeek即将涨价，但第三方工程师认为，这更可能是流量调控而非亏损压力。凭借极低缓存成本、极高缓存命中率以及架构优化，DeepSeek即使大幅提价仍具备价格优势。其真正护城河不仅在模型本身，也在缓存机制、基础设施和围绕模型构建的Harness体系。

第三方平台 OpenCode 工程师 dax 近日发帖讨论了 DeepSeek 即将涨价的事，他表示找到了即便是租赁 GPU，也能够复现其当前价格的方法。并且，**他推测这次涨价很可能并不是因为 DeepSeek 现在“亏钱”，而更像是一种流量调控手段，因为当前服务负载已经过高，他们希望通过提高价格来压低需求、缓解过载。**”



Dax 的猜测与很多网友感受一致。“这波主要还是 V4 Flash 整体性价比过于爆炸了，导致 0731 以来东大西大两边都在猛蹭，服务器给蹭冒烟了，什么峰谷定价，不存在的好吧，你这边的谷刚好就是那边的峰，24 小时全是高峰。所以大模型领域，高智、低价和服务的稳定性目前来讲还是个不可能三角。”有网友在知乎上说道。

对于 OpenCode 可以维持 DeepSeek 当前价格的消息，有网友评价，“如果属实，那会有很大的意义，也会让外界重新审视 DeepSeek 那支一直被认为是‘传奇’的强大基础设施团队。”

随后，dax 透露这并不是他们自己团队复现的，而是与其正在合作的另一支团队做到的。OpenCode 目前正在推动通过 OpenCode Go 在中国以外的地区使用低价 DeepSeek。

第三方 Token 更便宜，但花的钱仍比官方多

DeepSeek 以低价著称，但同一款 DeepSeek 模型，在第三方平台的价格甚至可能比 DeepSeek 自己更便宜。

OpenRouter 目前列出的 DeepSeek V4 Flash 0423 供应商数据显示，DeepInfra 对该模型的输入和输出价格分别为每百万 Token 0.09 美元和 0.18 美元，而 DeepSeek 官方对应价格为 0.14

美元和 0.28 美元。按标价计算，DeepInfra 大约只有 DeepSeek 官方价格的 64%，便宜约 36%。其他如 GMICloud、百度千帆等第三方平台的报价同样低于 DeepSeek 官方。

Provider	Input	Output	Cacheread	Latency	Throughput	Uptime
DigitalOcean	\$0.084	\$0.168	\$0.0168	1.20s	14 tps	99.51%
StreamLake	37% off \$0.08806	\$0.1761	\$0.01761	1.66s	34 tps	99.29%
Baidu Qianfan	37% off \$0.0882	\$0.1764	\$0.01764	0.91s	84 tps	99.81%
DeepInfra	\$0.09	\$0.18	\$0.018	2.46s	19 tps	98.89%
GMICloud	33% off \$0.0938	\$0.1876	\$0.01876	2.64s	40 tps	96.71%
SiliconFlow	\$0.13	\$0.28	\$0.028	1.42s	40 tps	97.07%
AlibabaCloud Int.	\$0.134	\$0.268	\$0.0268	1.00s	65 tps	96.52%
Venice	\$0.138	\$0.275	\$0.028	2.31s	24 tps	99.40%
Morph	\$0.139	\$0.278	\$0.07	1.62s	11 tps	94.25%
Parasail	\$0.14	\$0.28	\$0.07	0.65s	63 tps	97.94%
Fireworks	\$0.14	\$0.28	\$0.028	1.01s	66 tps	99.17%
Ambient	\$0.14	\$0.28	\$0.028	--	--	--
Cloudflare	\$0.14	\$0.28	\$0.028	0.85s	34 tps	100.00%
AtlasCloud	\$0.14	\$0.28	\$0.028	1.26s	47 tps	98.79%
OpenInference	\$0.14	\$0.28	\$0.028	3.00s	11 tps	98.62%
CoreWeave	\$0.14	\$0.28	\$0.07	0.45s	57 tps	98.60%
Mancer	\$0.20	\$0.50	--	1.20s	29 tps	96.68%
NovitaAI	\$0.14	\$0.28	\$0.028	1.25s	63 tps	98.30%
DeepSeek	\$0.14	\$0.28	\$0.0028	0.92s	91 tps	95.77%
Phala	\$0.20	\$0.40	\$0.07	2.26s	10 tps	93.96%

价格对比表，来源：OpenRouter

不过需要注意的是，DeepInfra 当前明确标注自己的 V4-Flash 为 FP4，而 DeepSeek 官方开放的模型仓库里包含 BF16、FP8 等，官方 API 究竟采用什么精度、什么推理策略，并没有在价格页中完整公开。

DeepSeek 已经公开 V4-Flash 权重，许可证允许第三方使用、修改、再分发甚至商业销售。因此，第三方厂商实际上并不是像以 0.14 美元买 DeepSeek API，再以 0.09 美元亏本转卖的方式运营，它们可以直接下载权重，在自己的 GPU 或其他加速器集群上运行模型。

这与闭源的 GPT、Claude 完全不同。第三方要卖 Claude，本质上通常仍然要向 Anthropic、AWS Bedrock 或 Google Vertex 购买 Claude Token。但第三方卖 DeepSeek V4，可以自己生产 Token。

但把价格与官方打平甚至更低后，用户花的钱会更少吗？

以 OpenCode Go 为例，其定位为面向开放模型的低成本 AI 编程订阅服务，首月价格 5 美元，之后为每月 10 美元，目前可以调用 DeepSeek V4 Pro、DeepSeek V4 Flash、GLM-5.2、Kimi K3、MiMo-V2.5、MiniMax M3、Qwen3.8 Max 等多款模型。OpenCode 官方目前给出的限制为每 5 小时最多 12 美元使用额度、每周 30 美元、每月 60 美元。官方还表示对于大多数模型，希望通过批量采购和预留 GPU 容量等方式，让用户以 10 美元订阅获得最高约 6 倍的模型使用价值。

正是“10 美元换 60 美元”让不少用户形成了一个直觉：既然 OpenCode Go 购买模型资源的实际折扣如此高，那么通过 Go 调用 DeepSeek 应该明显比直接购买 DeepSeek API 划算。

但事实并非如此。



一名开发者用真实 AI Coding 任务做了一次验证。他选取了一个包含 100 多个代码仓库的代码库，希望模型调查并理解其中某一特定领域的系统知识，然后同时打开两个终端，一个使用 OpenCode Go，一个直接连接 DeepSeek 官方 API，两边输入相同 Prompt、执行相同代码调查任务。

按照其公布的测试结果，在多个短任务中，OpenCode Go 显示的消费金额平均约为官方 DeepSeek API 的 4 倍；部分较长会话的差距超过 10 倍，也有少数任务只有约 2 倍。

这名开发者表示，他日常开发主要使用 Claude 和 OpenAI 订阅服务，每项月支出约 100 美元。DeepSeek V4 发布后，他开始通过官方 API 把 V4 Pro 以及部分 V4 Flash 用于一些规模较小的代码调查、可观测性分析以及需要快速获得结果的任务，先让 DeepSeek 搜索代码、读取文件、梳理问题，得到满意结果后再生成笔记，随后把这些结果交给 Claude 或 GPT 完成正式开发。按照他的使用方式，即便进行大量工作，一个月的 DeepSeek API 支出仍然可以控制在 10 美元以内。

不过有网友指出，OpenCode Go 界面里的“美元”与用户钱包里的美元，并不能直接画等号。其认为，Go 本质上是一个固定月费换取内部使用额度的订阅套餐，如果用户每月实际只支付 10 美元，那么界面显示消耗了数美元模型额度，并不意味着用户真的又支付了这笔钱。因此，将 OpenCode 显示的 8 美元额度消耗与 DeepSeek API 实际扣除的 2 美元直接比较，很可能夸大 OpenCode Go 的真实现金成本。但官方更便宜，确实是很多开发者的实际感受。

DeepSeek 的低价护城河

“主要差异似乎出在缓存命中率上。”该开发者表示，“DeepSeek 官方 API 在缓存处理方面明显比 OpenCode Go 更好。会话持续得越久，OpenCode Go 累积的缓存未命中似乎就越多，最终成本被放大的倍数也会越来越高。”

最终他得出结论：对于短任务来说，OpenCode Go 可能依然很方便。但如果是更长的会话，尤其是多轮交互累计的模型运行时间超过大约 5 分钟，那么通过 OpenCode Go 使用 DeepSeek，最终支付的成本可能会明显高于直接调用官方 API。

该开发者还提到了一个细节：使用 GLM 时，他可以通过模型专属设置提高缓存命中率，这类设置有点类似于配置 temperature 或其他请求参数。具体来说，可以设置模型不要重写之前的消息，也不要删除前几轮中的推理内容。但是，OpenCode 并没有为 DeepSeek 暴露类似的模型设置。

对于 Coding Agent，模型需要不断读取代码、调工具、返回结果，然后将已经积累的大量上下文再次带入下一轮推理。如果历史 Prompt 能够被缓存，后续请求中的大量重复上下文就可以按照极低的 Cache Hit 价格计费；一旦 Prompt 前缀发生变化导致缓存失效，几十万甚至更多历史 Token 就可能重新按照普通输入价格处理。随着会话越来越长，这种差异会迅速放大。

这也是为什么现在单以“每百万 Token 多少钱”来衡量成本高低，正在变得越来越不准确。

在上面价格对比表中，真正值得注意的是 DeepSeek 官方的“0.0028 美元缓存价”，这是列表中的最低价格。如果能做到 95%、99% 的缓存命中，DeepSeek 官方的 0.0028 美元几乎相当于白送输入 Token。但之前有用户测试，DeepSeek 可以达到这个命中率。

知乎答主“苏迟但到”，进行了一次两万次实验、持续 3 天的测试，覆盖 DeepSeek、Kimi、智谱 GLM、MiniMax 以及通过 OpenRouter 调用的 OpenAI 模型。具体方法是，每隔 40 分钟向每



个模型发送 27 条随机请求，并在 1 分钟至 720 分钟、也就是最长 12 小时后再次发送相同内容，记录缓存是否仍然命中，以减少系统负载波动带来的偶然性。

结果显示，DeepSeek 表现最突出。无论工作时间还是非工作时间，其缓存命中率始终保持 100%，即使间隔达到 12 小时仍然没有出现明显失效。MiniMax 排名第二，非工作时间命中率约 90%，工作时间则下降至约 70%。Kimi 和 OpenAI 表现居中，而 GLM 最弱，2 分钟后命中率约 80%，3 分钟降至 50%，5 分钟仅剩 25%，几乎没有观察到超过 15 分钟的缓存存活。

该答主认为，GLM 表现较弱可能有两种原因：一是 KV Cache 更多依赖显存，缺乏向更低成本存储介质持久化的能力，导致缓存频繁淘汰；二是实际请求量过大，超过可缓存容量，从而加速旧缓存失效。不过，这两种解释目前都只是推测。

那么，我们以 OpenRouter 给出的统一口径下的缓存命中率来计算，假设 DeepSeek 官方缓存命中率达到 78.2%（实际上要更高），缓存价格为第三方约一成、其他价格相同，那么 DeepSeek 在 Cache Hit、Cache Miss 全部同比提价约 3 倍才会与第三方平台基本持平；而如果只提高 Cached Read，普通输入价不变的话，那提高 30 倍左右才会与第三方平台基本持平。

Provider	Input	Output	Cache hit rate	Token share
NovitaAI	\$0.04532	\$0.279	84.5%	24.3%
DeepSeek	\$0.03271	\$0.2794	78.2%	16.2%
StreamLake	\$0.04356	\$0.1751	63.1%	14.5%
GMICloud	\$0.03704	\$0.1869	75.6%	14.2%
DeepInfra	\$0.04889	\$0.1766	56.9%	7.4%
Alibaba Cloud Int.	\$0.06744	\$0.2673	62.0%	5.7%
Parasail	\$0.1141	\$0.2794	37.0%	3.4%
DigitalOcean	\$0.06458	\$0.1665	28.8%	3.2%
CoreWeave	\$0.122	\$0.2787	25.6%	2.3%
Cloudflare	\$0.116	\$0.2788	21.3%	1.9%
SiliconFlow	\$0.08756	\$0.279	41.5%	1.8%
AtlasCloud	\$0.104	\$0.2793	32.1%	1.5%
Venice	\$0.09607	\$0.2743	38.0%	1.5%
Morph	\$0.1064	\$0.2767	47.2%	0.7%
Fireworks	\$0.1089	\$0.2794	27.7%	0.7%
OpenInference	\$0.1399	\$0.2785	0.0%	0.3%
Baidu Qianfan	\$0.07867	\$0.1757	13.4%	0.2%
Mancer	\$0.1996	\$0.4986	0.0%	0.1%
Phala	\$0.1935	\$0.399	4.9%	0.0%

来源：OpenRouter

这种优势得益于 DeepSeek 的架构设计。

最新官方缓存文档披露了其三种缓存前缀落盘机制。第一种是在每次请求的用户输入结束位置和模型输出结束位置自动生成缓存前缀；第二种是公共前缀检测（Common Prefix Detection），如果系统发现多次请求存在共同前缀，会主动把这部分公共前缀单独落盘；第三种是针对超长输入和输出，按照固定 Token 间隔主动生成缓存前缀单元，避免必须等到一个超长请求结束后才能形成可复用缓存。其中，“公共前缀检测”方式可以直接提高实际命中概率。而不再使用的缓存一般会在数小时到数天后清除。

到了 V4，官方采用“Token-wise Compression + DeepSeek Sparse Attention（DSA）”，核心目的就是进一步降低百万 Token 上下文的计算和内存成本，并把 100 万 Token 上下文变成所有官



方服务的标准配置。

根据 vLLM 团队对 V4 架构的拆解，V4 不仅共享 Key/Value，还会跨 Token 进一步压缩 KV Cache，其中一种模式大约压缩 4 倍，另一种可达到约 128 倍。在百万 Token 上下文下，其估算的 BF16 KV Cache 约为 9.62 GiB，而 V3.2 结构约为 83.9 GiB，缩小大约 8.7 倍；实际部署时又使用 FP4 保存 Indexer Cache、FP8 保存 Attention Cache，还能再把 Cache 占用压低大约一半。

但这个优势能持续多久，或许也是一个值得思考的问题。

今天，dax 发帖子称，过去 48 小时里，DeepSeek 流量比任何其他平台都多。这些流量来自各种不同的客户端，并不只是 OpenCode。他给了一些平台的缓存命中率：

CLIENT	CACHE HIT RATIO
ZCode	98.60%
OpenCode V2	97.86%
Cursor	97.84%
Kimi Code CLI	97.64%
Pi	96.98%
Codex	96.66%
Kilo Code	95.73%
OpenCode V1	95.69%
Claude Code / CLI	89.31%

不过，第三方和官方在质量上也会有差异。

有开发者反复测试得出，经 OpenCode 路由的 DeepSeek V4 Flash，上下文窗口其实只有 600k 左右，超过以后 OpenCode 会作某种压缩截断。而官网的是全量的 1M 上下文，所以能力更全面，性能更加强大。“很多人对 DeepSeek 能力的评价区别很大，可能也有这个原因。”

Harness 或比智力更重要

V4-Flash-0731 没有扩充基础模型规模，仍然是原来的模型架构，主要变化来自重新后训练以及 Agent 适配。也就是说，DeepSeek 在展示“V4 Flash 为什么突然拥有这么强的 Coding Agent 能力”时，实际上把两个因素绑定在了一起：模型后训练 + DeepSeek 自己的 Harness。

该开发者在实际使用中发现，同一款 DeepSeek V4 Flash，在不同 AI Coding 工具中的表现可能存在明显差异。其体验是，DeepSeek V4 Flash 在 Codex 中的实际水平可以与 Fable 5 基本打平，但放到 OpenCode 环境中，只能发挥出大约一半的真实水平。

这种差异主要体现在两个方面。首先是对问题的全面理解能力。在 Codex 中，DeepSeek V4 Flash 对代码的读取以及对整体问题的宏观把握明显更好，不容易出现只关注局部问题、忽略整个系统的情况。相比之下，在 OpenCode 中运行时，模型更容易陷入局部细节，影响对整体任务的判断。



第二个差异则出现在长程任务中的记忆连贯性。复杂的软件开发任务通常包含大量彼此关联的问题，局部最优解并不一定意味着全局最优。一个局部问题得到解决后，可能引发更多全局问题，最终使 Agent 不断在不同方案之间循环，甚至在任务末尾给出实际上并不成立的解决方案。开发者表示，这类问题在 OpenCode 中的 DeepSeek 上仍然较为常见。

但在 Codex 中，DeepSeek V4 Flash 却很少出现类似情况。即使任务执行链条已经很长，模型仍然能够保持对最初目标的理解，不容易因为连续解决大量局部问题而偏离整个任务方向，可以说是“走得再远，也不会忘记为什么出发”。

实际上，DeepSeek 官方此前有发布一份 Codex 接入文档，直接提供了一整套models.json模型配置，让 Codex 知道应该怎样“驾驭”V4 Flash。

这份配置实暴露出了一些 Harness 层面的适配信息。例如 V4 Flash 在 Codex 里被配置为支持并行工具调用、约 104 万 Token 上下文、95% 的有效上下文窗口，并定义了自由形式的 apply_patch 工具、Web Search 等，文档甚至明确了涉及长任务中的 compaction 行为，也就是上下文过长之后如何压缩历史、怎样在压缩后继续任务而不是从头开始等。

这也让该开发者把关注点从模型本身进一步转向 Harness。一个成熟的 Harness 背后涉及消息路由、记忆管理、长短期记忆的优化与调用、工具调用等大量机制，这些能力往往来自长期、反复的工程试验和优化，而且相关 SDK 并不会完整公开所有实现细节和技巧。

因此，在其看来，未来 AI Coding 产品之间的竞争可能不再只是模型智力高低。围绕模型建立起来的 Harness 体系，以及其中沉淀的消息路由、记忆管理和工具调用能力，可能成为比模型本身更深的一层护城河。

本文来源：[InfoQ](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码

VIP会员·7天畅读卡 | 大师课·入门串讲

免费领取



限免

281

收藏

分享到:  

写评论

请发表您的评论

表情

图片

发表评论

1 条评论





MobileUser2798

15小时前 中国

牛的。芯片半导体快涨

👍 0 ↩ 回复

